VERIFAI

AI-POWERED

Fake News Detection System

An NLP-based system to detect misinformation using machine learning and text analysis

NLP | scikit-learn | FastAPI

Project by:
Group: Algorix
•Ayana Roy • Shrijeeta Das

99.63%

Model Accuracy

OVERVIEW

Presentation Agenda

01 Introduction
Background, motivation & objectives

02 Problem Statement
The challenge of misinformation at scale

03 Literature Survey
Prior work and comparison of approaches

04 Proposed Solution
Our NLP pipeline and architecture

05 Dataset & Model
ISOT dataset, TF-IDF, PAC training

06 Implementation
FastAPI backend, web interface, code walkthrough

07 Results
99.63% accuracy, metrics, confusion matrix

08 Future Scope
BERT, browser extension, multi-language support

09 Conclusion
Key takeaways and summary

Background & Motivation

What is Fake News?

Fake news refers to fabricated or deliberately misleading content presented as factual journalism. It is designed to deceive readers for political, financial, or ideological purposes.

Why Does It Matter?

- Influences election outcomes and political decisions globally
- Erodes public trust in institutions and legitimate journalism
- Caused vaccine hesitancy and health crises during COVID-19
- Spreads 2-3x faster than real news on social media platforms
- Manual fact-checking cannot scale to millions of daily posts

6 in 10

Americans encounter fake news regularly

$78B

Annual global economic damage from fake news

2-3×

Faster spread than true news online

Objectives & Scope

Project Objectives

Design & Train

Build an NLP binary classifier using TF-IDF + Passive Aggressive Classifier

High Accuracy

Achieve near state-of-the-art accuracy on a real-world labeled news dataset

Confidence Score

Provide probability-like confidence scores alongside every prediction

Web Deployment

Deploy the model through a FastAPI REST API with an HTML web interface

Extensibility

Build a modular system that can be upgraded with BERT or other models later

Scope & Boundaries

Language

English-language news articles and social media posts

Domain

Political news — highest prevalence of misinformation

Output

Binary: FAKE (0) or REAL (1) with confidence score

Input

Headline + body text combined as single feature string

Platform

Web-based; accessible via browser at localhost:8000

Model Size

~45MB model.pkl — deployable on any standard laptop

Why Fake News Detection Matters

6 in 10

Americans encounter fake news regularly

2–3×

Faster spread of false news vs. true news

$78B

Annual economic damage from misinformation

Key Challenges

- Manual fact-checking cannot scale to millions of posts generated every minute
- Modern fake news mimics legitimate journalism — hard even for humans to detect
- Misinformation spreads faster than corrections, causing irreversible damage
- Existing social media platforms lack real-time automated detection tools

Formal Problem Definition

Given: A text T (news article or social media post)

Classify: T as either FAKE (0) or REAL (1)

Output: Confidence score C in [50%, 99%]

Constraint: Real-time inference, no GPU required

Prior Work & Research Background

Shu et al., 2017

TF-IDF + SVM

Pioneered bag-of-words ML approach. Achieved ~70% accuracy. Established n-gram features as key signals for fake news classification.

Wang, 2017

LIAR Dataset

Introduced 12,836 labeled statements. LSTM networks achieved ~27% on 6-class credibility task, revealing fine-grained classification difficulty.

Ahmed et al., 2018

Ensemble ML

Voting ensemble of Naive Bayes, SVM, Decision Trees on ISOT dataset showed 3-5% improvement over single classifiers, reaching ~96% accuracy.

Kula et al., 2020

BERT Fine-tune

Transformer models (BERT, RoBERTa) achieved 95-97% accuracy but require GPU hardware and are too slow for real-time deployment.

Approach Comparison

Approach	Accuracy	Speed	Resources	Suitable for Real-time?
TF-IDF + SVM	~94%	Fast	Low	Yes
TF-IDF + PAC (Ours)	**99.63%**	**Very Fast**	**Low**	**Yes**
LSTM Neural Network	~85-90%	Medium	Medium	Partial
BERT Transformer	~97%	Slow	High (GPU)	No
Ensemble (NB+SVM+DT)	~96%	Fast	Low-Medium	Yes

Our approach achieves the highest accuracy while remaining fast and resource-efficient

Our Approach: NLP Pipeline

01
Input
News article or headline

02
Preprocess
Title + Body combined text

03
TF-IDF
50K-feature vectorization

04
Classify
Passive Aggressive Classifier

05
Output
FAKE / REAL + confidence %

Why TF-IDF + Passive Aggressive Classifier?

No GPU needed
Runs on any laptop — ideal for academic & small-scale deployment

Fast training
Trains on 35,918 articles in under 30 seconds on standard hardware

Online learning
PAC updates only on mistakes — memory-efficient for high-dimensional data

Proven accuracy
Achieves 99.63% — outperforms LSTM and matches BERT without compute cost

Interpretable
TF-IDF features are human-readable; no black-box neural network required

Production-ready
Serialized pipeline (~45MB) loads in 2 seconds; inference in <50ms

System Architecture

User Browser

http://localhost:8000 • HTML/JS UI

FastAPI Server

main.py • GET / and POST /predict routes

Jinja2 Template

template/index.html • Web interface

ML Pipeline

model/model.pkl • TF-IDF + Classifier

JSON Response

{ "label": "FAKE", "confidence": 97.4 }

Project File Structure

```
FakeNewsDetector/
  ├── data/
  |   ├── Fake.csv
  |   └── True.csv
  ├── model/
  |   ├── train.py
  |   └── model.pkl
  ├── template/
  |   └── index.html
  └── main.py
```

Dataset & Preprocessing

ISOT Fake News Dataset

University of Victoria • 2015-2018

Total Samples	44,898 articles
Fake Articles	23,481 (52.3%)
Real Articles	21,417 (47.7%)
Train Split	35,918 (80%)
Test Split	8,980 (20%)
Source - Fake	Unreliable websites
Source - Real	Reuters News Agency
Text Field	title + body combined

Preprocessing Pipeline

1. **Load Data**
   pd.read_csv() for Fake.csv and True.csv
2. **Label**
   Assign label=0 (Fake) and label=1 (Real)
3. **Shuffle**
   Concatenate & shuffle with random_state=42
4. **Combine Text**
   content = title.fillna('') + ' ' + text.fillna('')
5. **Drop Nulls**
   Remove rows where combined content is empty
6. **Split**
   train_test_split(X, y, test_size=0.2, random_state=42)
7. **Vectorize**
   TfidfVectorizer transforms text to 50K-dim sparse matrix

Model Training & Configuration

TF-IDF Vectorizer Parameters

stop_words	"english"	Remove common words like 'the', 'is'
max_df	0.7	Ignore words in >70% of documents
ngram_range	(1, 2)	Use unigrams + bigrams for richer features
max_features	50,000	Cap vocabulary size for memory efficiency
sublinear_tf	True	Log normalization: 1 + log(tf)

PassiveAggressiveClassifier

max_iter	50	Training iterations
random_state	42	Reproducibility seed

Model Performance

99.63% Overall Accuracy	**99%** FAKE Precision
100% REAL Precision	**100%** FAKE Recall
99% REAL Recall	**1.00** F1-Score

Training Time: ~25 seconds • Model Size: ~45MB • Inference: <50ms

Code Walkthrough

model/train.py — Key Snippet

```
pipeline = Pipeline([
  ("tfidf", TfidfVectorizer(
      stop_words='english',
      max_df=0.7, ngram_range=(1,2),
      max_features=50000,
      sublinear_tf=True)),
  ("clf", PassiveAggressiveClassifier(
      max_iter=50))
])
pipeline.fit(X_train, y_train)
joblib.dump(pipeline, "model.pkl")
```

Web Interface Features

- Paste any news article into the text area
- Click 'Analyze' — result shown instantly
- Color-coded FAKE (red) / REAL (green) badge
- Animated confidence bar (50-99% range)
- Sample buttons to test with preset articles

main.py — Prediction Endpoint

```
@app.post('/predict')
async def predict(news: NewsInput):
    pred = model.predict([news.text])[0]
    d    = model.decision_function(
               [news.text])[0]
    conf = 1/(1+np.exp(-abs(d)))*100
    return {
      "label": "REAL" if pred==1
              else "FAKE",
      "confidence": round(conf, 2)}
```

How to Run the System

1. `py -3.11 -m pip install fastapi uvicorn scikit-learn`
2. `cd model && py -3.11 train.py`
3. `py -3.11 -m uvicorn main:app --reload`
4. Open browser → http://localhost:8000

Model Results & Evaluation

99.63%	44,898	~50K	<50ms
Overall Accuracy	Training Samples	TF-IDF Features	Inference Time

Confusion Matrix (8,980 test samples)

	Predicted FAKE	Predicted REAL
Actual FAKE	4,693 (TP)	17 (FN)
Actual REAL	16 (FP)	4,254 (TN)

Classification Report

Class	Precision	Recall	F1
FAKE	0.99	1.00	1.00
REAL	1.00	0.99	1.00
Accuracy			0.9963
Macro avg	1.00	1.00	1.00

Enhancements & Roadmap

Deep Learning

Integrate BERT / RoBERTa transformers for better contextual understanding and semantic analysis of subtle misinformation.

Browser Extension

Chrome/Firefox extension to analyze articles inline as users browse, without copy-pasting into a separate tool.

Multi-language

Support Hindi, Bengali, and other regional languages using multilingual models (mBERT, XLM-RoBERTa).

Source Credibility

Incorporate domain reputation scores and cross-reference with fact-checking APIs (Snopes, PolitiFact).

Real-time Monitoring

Integrate Twitter/X API to monitor trending content and flag viral misinformation before it spreads widely.

Explainability (XAI)

Add LIME/SHAP-based word-level explanations to show users which phrases triggered the classification.

Key Takeaways

What We Built & Achieved

- Trained an NLP pipeline on 44,898 real-world news articles from scratch
- Achieved 99.63% accuracy — outperforming LSTM and matching BERT without GPU
- Deployed a live REST API using FastAPI, served via Uvicorn ASGI server
- Built a clean web UI with color-coded verdict and animated confidence score
- Entire system runs on a standard laptop using open-source tools only
- Model file is ~45MB and loads in under 2 seconds at server startup

Project Summary

99.63% Accuracy	**44,898** Articles
<50ms Inference	**7 files** Full System
0 GPU Required	**Free** Open Source

Thank You

VERIFAI: AI-Powered Fake News Detection System

Accuracy: 99.63% • Dataset: 44,898 articles • Stack: Python, TF-IDF, FastAPI

NLP | scikit-learn | FastAPI | Python 3.11 | REST API